

Especificación de la clase SubscriptionService

Repositorio (formato Maven)

Arquitectura del sistema

La clase `SubscriptionService` implementa un servicio de envío de mensajes a los usuarios de una aplicación.

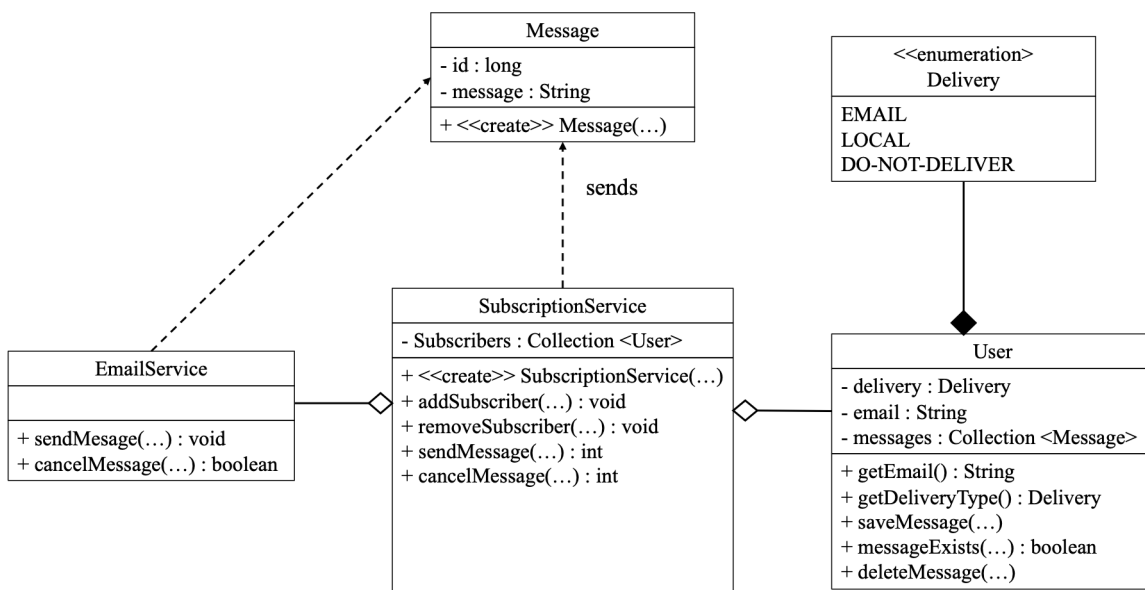


Figura 1. Diagrama de clases

La arquitectura del sistema software se muestra en la **Figura 1**. `SubscriptionService` permite suscribir usuarios a un servicio de mensajería. Los mensajes se envían por email mediante la clase `EmailService`.

Los usuarios pueden especificar un `Delivery` de tipo `LOCAL`, lo que implica que los mensajes se almacenan en la clase `User` para su posterior consulta, y no son enviados por

email. También existe un `Delivery` de tipo `DO_NOT_DELIVER`, lo que permite a los usuarios no recibir mensajes, pero permanecer suscritos.

El envío de mensajes puede cancelarse en algunos casos. Cuando los correos son enviados por email, `EmailService` establece un retraso de entrega que permite cancelar un mensaje por tiempo limitado. En el caso de un `Delivery` de tipo `LOCAL`, el mensaje se elimina de la clase `User`.

Finalmente, `SubscriptionService` también permite eliminar la suscripción de un usuario al servicio de mensajes.

Las clases `EmailService` está siendo implementada por un equipo distinto y no puede ser modificada. Tampoco se deben realizar cambios en las clases `User`, `Message` y `Delivery`.

Implementación de la clase `SubscriptionService`

La clase `SubscriptionService` **toma los siguientes valores durante la inicialización:**

- ☐ La lista de usuarios suscritos al servicio de mensajería debe estar vacía.

Se deben implementar los siguientes métodos de la clase `SubscriptionService`:

- `void addSubscriber(User user)`

Este método añade un usuario (`user`) al servicio de mensajería.

- ☐ Se deberá añadir `user` a la lista de suscriptores del servicio de mensajería.
 - ☐ Si `user` es `null`, se deberá generar una excepción `NullUserException`.
 - ☐ Si `user` ya está suscrito, se deberá generar una excepción `ExistingUserException`.
 - ☐ Si el email asociado a un `user` es `null`, se deberá generar una excepción `UserDoesNotHaveEMailException`.
 - ☐ Si `Delivery` es de tipo `LOCAL`, no se verifica el email del `user`.

- `void removeSubscriber(User user)`

Este método elimina un usuario (`user`) del servicio de mensajería.

- ☐ Se deberá eliminar `user` de la lista de suscriptores del servicio de mensajería.
 - ☐ Si `user` es `null`, se deberá generar una excepción `NullUserException`.
 - ☐ Si `user` no está suscrito, se deberá generar una excepción `NonExistingUserException`.
- `int sendMessage(Message message)`

Envía un mensaje a todos los usuarios suscritos.

- ☐ `message` es enviado a todos los `User` suscritos usando `EmailService`.
 - ☐ Cuando `user` tiene asignado un `Delivery` de tipo `LOCAL`, `message` es almacenado en la lista `messages` de `user`.
 - ☐ Cuando `user` tiene asignado un `Delivery` de tipo `DO_NOT_DELIVER`, el `user` recibe un mensaje con el texto "Ha perdido ud. un mensaje" en la lista `messages` de `user`. Este nuevo mensaje tiene el mismo `id` que el mensaje original.
- ☐ El método `sendMessage()` devuelve el número de mensajes descartados.